

Function Arguments

```
In [ ]: 1.Positional Arguments
        2.Keyword Arguments
        3.Default Arguments
        4.Arbitrary Positional Arguments.
        5.Arbitrary Keyword Arguments
```

1.Positional Argumnets

```
In [ ]: Number of parameter==Number of Arguments
```

```
In [1]: def calculation(a,b):
        print(f"a=={a} and b=={b}")
        add=a+b
        mul=a*b
        return add,mul
        calculation(10,20)
```

a==10 and b==20

Out[1]: (30, 200)

```
In [2]: def calculation(a,b):
        print(f"a=={a} and b=={b}")
        add=a+b
        mul=a*b
        return add,mul
        calculation(10)
```

TypeError

Traceback (most recent call last)

Cell In[2], line 6

4 mul=a*b

5 return add,mul

----> 6 calculation(10)

TypeError: calculation() missing 1 required positional argument: 'b'

```
In [3]: def calculation(a,b):
        print(f"a=={a} and b=={b}")
        add=a+b
        mul=a*b
        return add,mul
        calculation(10,20,30)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[3], line 6
      4     mul=a*b
      5     return add,mul
----> 6 calculation(10,20,30)
```

TypeError: calculation() takes 2 positional arguments but 3 were given

2.Keyword Arguments

In []: Keyword argument follows positional arguments.

```
x,y,z=10 #valid
x=10,20,30 #invalid
```

```
In [4]: def addition(a,b,c):
        print("This is addition Function")
        add=a+b+c
        print(f"Addition of numbers is:",add)
        return add
        addition(10,20,30)
```

```
This is addition Function
Addition of numbers is: 60
```

Out[4]: 60

```
In [5]: def addition(a,b,c):
        print("This is addition Function")
        add=a+b+c
        print(f"Addition of numbers is:",add)
        return add
        addition(a=10,b=20,c=30)
```

```
This is addition Function
Addition of numbers is: 60
```

Out[5]: 60

```
In [6]: def addition(a,b,c):  
        print("This is addition Function")  
        add=a+b+c  
        print(f"Addition of numbers is:",add)  
        return add  
        addition(a=10,c=20,b=30)
```

This is addition Function
Addition of numbers is: 60

Out[6]: 60

```
In [11]: def addition(a,b,c=50):  
         print("This is addition Function")  
         add=a+b+c  
         print(f"Addition of numbers is:",add)  
         return add  
         addition(a=10,b=20)
```

This is addition Function
Addition of numbers is: 80

Out[11]: 80

```
In [12]: def addition(a,b,c=50):  
         print("This is addition Function")  
         add=a+b+c  
         print(f"Addition of numbers is:",add)  
         return add  
         addition(a=10,20)
```

Cell In[12], line 6
 addition(a=10,20)
 ^

SyntaxError: positional argument follows keyword argument

```
In [13]: def addition(a,b,c=50):  
         print("This is addition Function")  
         add=a+b+c  
         print(f"Addition of numbers is:",add)  
         return add  
         addition(a=10,b=20,c=60) #you can change the value at the time calling
```

This is addition Function
Addition of numbers is: 90

Out[13]: 90

Default Arguments

In []: In a function arguments can have default values to the arguments using the '=' (assignment Operator) at the time of function definition

```
In [2]: def Student(name,age,grade='Five'):
        print("Student Details:",name,age,grade)

        Student('John',12)
        Student('Ram',13)
        Student('Sham',14,'Six')
```

```
Student Details: John 12 Five
Student Details: Ram 13 Five
Student Details: Sham 14 Six
```

```
In [3]: def Student(name,age,grade='Five'):
        print("Student Details:",name,age,grade)

        Student('John',12)
        Student('Ram',13)
        Student('Sham',14,grade='Six')
```

```
Student Details: John 12 Five
Student Details: Ram 13 Five
Student Details: Sham 14 Six
```

4)Arbitrary Positional Arguments(*args)

In []: We can pass multiple arguments to the function. Internally all these values are represented in the form tuple.

```
In [4]: def average(*args):
        sum=0
        for i in args:
            sum+=i
        avg=sum/len(args)
        print(avg)
        average(2,3,4,5,6,7)
```

```
4.5
```

5)Arbitrary Keyword Arguments(**kwargs)

In []: The **kwargs allow you to pass multiple keyword arguments to the function use the **kwargs. If you want to handle named arguments in function.

```
In [7]: def test_arguments(a,b,c,*args,x=100,y=200,**kwargs):
        print("Positional Arguments:",a,b,c)
        print("Arbitrary Positional Arguments:",args)
        print("Default Arguemtns:",x,y)
        print("Arbitrary Keywprd Arguments:",kwargs)
test_arguments(4,5,6,7,9,8,11,12,y='Python',x=500,phy=98,chem=88,math=89,bio=67)
```

Positional Arguments: 4 5 6
 Arbitrary Positional Arguments: (7, 9, 8, 11, 12)
 Default Arguemtns: 500 Python
 Arbitrary Keywprd Arguments: {'phy': 98, 'chem': 88, 'math': 89, 'bio': 67}

```
In [8]: def test_arguments(a,b,c,*args,x=100,y=200,**kwargs):
        print("Positional Arguments:",a,b,c)
        print("Arbitrary Positional Arguments:",args)
        print("Default Arguemtns:",x,y)
        print("Arbitrary Keywprd Arguments:",kwargs)
test_arguments(4,5,6,7,9,8,11,12,y='Python',phy=98,chem=88,math=89,bio=67)
```

Positional Arguments: 4 5 6
 Arbitrary Positional Arguments: (7, 9, 8, 11, 12)
 Default Arguemtns: 100 Python
 Arbitrary Keywprd Arguments: {'phy': 98, 'chem': 88, 'math': 89, 'bio': 67}

Find out square of even and cube of odd number

```
In [11]: def get_even_odd(x):
        if x%2==0:
            return f"Square of {x} is {x**2}"
        else:
            return f"Cube of {x} is {x**3}"
get_even_odd(2)
```

Out[11]: 'Square of 2 is 4'

Recursive Function

In []: When function calls itself again again **is** called recursive function.

```
In [12]: def get_even_odd(x):
        if x%2==0:
            print(f"Square of {x} is {x**2}")
        else:
            return f"Cube of {x} is {x**3}"
a=get_even_odd(2)
a
```

Square of 2 is 4

Lambda Functions

In []: Anonymous Functions>>A function without name.

```
map(function_name,iterable)
filter(function_name,iterable)
```

No need to use **def** keyword and **return** statement.
Lambda functions are defined by using **lambda** keyword.

Syntax: **lambda** arguments:expression
Arguments>>Multiple
Expression>>Single

In [13]: `x=lambda string:string.upper()`
`print(x('Python'))`
`print(x('Data science'))`

PYTHON
DATA SCIENCE

Built Functions

1)filter

In []: We can **not** change value here.
Syntax:`filter(function_name,iterable)`

In [14]: `def get_odd_num(x):`
 `if x%2!=0:`
 `return x`
`list(filter(get_odd_num,[2,3,4,5,6,7]))`

Out[14]: [3, 5, 7]

In [15]: `string="Data Science"`
`list(filter(lambda x:x.islower(),string))`

Out[15]: ['a', 't', 'a', 'c', 'i', 'e', 'n', 'c', 'e']

In [16]: `string="Data Science"`
`list(filter(lambda x:x.isupper(),string))`

Out[16]: ['D', 'S']

2)map()

```
In [ ]: We can change value here.  
We can pass multiple arguments.  
Syntax: map(function_name, iterable)
```

```
In [17]: list1=[4,5,6,7,8]  
list2=[10,20,30,40,50]  
list(map(lambda x,y:x+y,list1,list2))
```

```
Out[17]: [14, 25, 36, 47, 58]
```

```
In [18]: list1=[4,5,6,7,8]  
list2=[10,20,30,40,50]  
list(map(lambda x,y:x*y,list1,list2))
```

```
Out[18]: [40, 100, 180, 280, 400]
```

```
In [19]: player_name=['Virat','Rohit','Hardik','Suryakumar','Sachin','kartik']  
result=list(filter(lambda x:len(x)==5,player_name))  
result
```

```
Out[19]: ['Virat', 'Rohit']
```

Write a python program to find square and cube every number in a given list of intergers using lambda

```
In [20]: list1=[1,2,3,4,5,6]  
squares=list(map(lambda x:x**2,list1))  
print(squares)  
cube=list(map(lambda x:x**3,list1))  
print(cube)
```

```
[1, 4, 9, 16, 25, 36]  
[1, 8, 27, 64, 125, 216]
```

```
In [ ]:
```